

Q11. Debugging Selection Sort (Incorrect Swap Position)

1. Problem Understanding & Context Analysis

Problem Statement

The given Selection Sort implementation performs swapping inside the inner loop whenever a smaller element is found. This causes unnecessary swaps and affects the efficiency of the algorithm. The task is to identify the logical error, correct the swapping mechanism, optimize the implementation, and analyze its complexity.

Understanding of the Problem

Program Objective:

The program should sort an array in ascending order using the Selection Sort algorithm.

Input: An unsorted array of integers.

Output: A sorted array in ascending order.

Algorithm Concept Involved: Selection Sort repeatedly finds the minimum element from the unsorted portion of the array and places it at its correct position.

Location of Issue:

The swap statement is placed inside the inner loop.

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Root Cause Analysis

S.No	Error Line	Type of Error	Explanation
1	Swap statement inside inner loop	Logical Error	Swapping occurs before the minimum element is fully identified.
2	Multiple unnecessary swaps	Performance Issue	Elements are exchanged many times unnecessarily.
3	Incorrect Selection Sort logic	Algorithm Error	Selection Sort should swap only after finding the minimum element.

Why the Error Occurs:

The algorithm swaps elements during every iteration of the inner loop instead of waiting until the minimum element is found.

Impact on Output:

- Produces unnecessary operations.
- Reduces efficiency.
- May disturb intermediate ordering.

2. Application of Concepts

Original Code

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ISSUE
    return arr
```

Identified Errors

S.No	Error Line	Type of Error	Explanation
1	Swap statement	Logical Error	Executed inside inner loop.
2	Repeated swapping	Performance Issue	Causes unnecessary operations.
3	Wrong implementation	Algorithm Error	Violates Selection Sort principle.

Corrected Logic

- Move swapping outside the inner loop.
- Perform swap only after the minimum element is found.
- Swap only if min_idx != i.

Algorithm Rule Followed: Selection Sort performs exactly one swap per pass after identifying the minimum element.

How Correctness is Restored: The smallest element is placed in its correct position only after complete comparison.

3. Analytical & Decision-Making Skills

Debugging Process

Step 1:

Identify that swapping occurs inside the inner loop.

Step 2:

Move the swap statement outside the inner loop.

Step 3:

Add a condition to avoid swapping when the minimum element is already in the correct position.

Optimization Applied

- Added:

```
if min_idx != i:
```

```
    before swapping.
```

Benefits

- Avoids unnecessary swaps.
- Reduces write operations.
- Improves practical performance.

Justification

The corrected version follows the standard Selection Sort algorithm exactly. The optimization prevents redundant swaps while maintaining correctness.

4. Solution Design & Approach**Corrected & Optimized Code:**

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_idx = i

        # Find minimum element
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap only when needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

Algorithm / Logic Flow

1. Start the program.
2. Read input array.
3. Assume first unsorted element as minimum.
4. Search remaining elements for smaller value.
5. Update minimum index if found.
6. Swap only if required.
7. Repeat until array is sorted.
8. Display sorted array.
9. Stop.

Output:

Output

```
Initial array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]
```

```
=== Code Execution Successful ===
```

5. Clarity, Justification & Presentation

Time Complexity Analysis

Case	Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity

- Auxiliary Space: **$O(1)$**

Final Conclusion

- Swapping inside the inner loop was the main logical error.
- Swapping was moved outside the inner loop.
- Additional condition prevents unnecessary swaps.
- Algorithm now correctly follows Selection Sort.
- Performance is improved by reducing redundant operations.

Q40. Debugging Insertion Sort Code (Incorrect Final Placement Index)

1. Problem Understanding & Context Analysis

Problem Statement

The given Insertion Sort implementation correctly shifts larger elements but inserts the key element at the wrong position after shifting. The task is to identify the indexing error, correct it, optimize the implementation, and analyze its complexity.

Understanding of the Problem

Program Objective: Sort an array in ascending order using Insertion Sort.

Input: An unsorted array of integers.

Output: Sorted array in ascending order.

Algorithm Concept Involved: Insertion Sort inserts each element into its proper position in the already sorted portion of the array.

Location of Issue: `arr[j] = key`

Root Cause Analysis

S.No	Error Line	Type of Error	Explanation
1	<code>arr[j] = key</code>	Logical Error	Inserts key at wrong index.
2	<code>j</code> becomes one position smaller	Indexing Error	After loop, <code>j</code> points before insertion position.
3	Incorrect placement	Algorithm Error	Sorted order gets disturbed.

Why the Error Occurs:

After shifting elements, `j` is decremented one extra time. Therefore the correct insertion position is `j + 1`.

Impact on Output:

- Key may overwrite an existing value.
- Array becomes incorrectly sorted.

2. Application of Concepts

Original Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1

        arr[j] = key # ERROR

    return arr
```

Identified Errors

S.No	Error Line	Type of Error	Explanation
1	arr[j] = key	Logical Error	Wrong insertion position.
2	j decrement	Indexing Issue	j points one position before insertion point.
3	Incorrect sorted prefix	Algorithm Error	Disturbs sorted order.

Corrected Logic

Replace:

arr[j] = key

with

arr[j + 1] = key

Algorithm Rule Followed:

After all larger elements are shifted, insert the key at position j + 1.

How Correctness is Restored:

The key is placed exactly where all previous elements are smaller and all following shifted elements are larger.

3. Analytical & Decision-Making Skills

Debugging Process

Step 1:

Trace variable j during shifting.

Step 2:

Observe that j decreases below the insertion position.

Step 3:

Insert key at $j + 1$ instead of j .

Optimization Applied

- Improved readability with comments.
- Used consistent variable naming.
- Maintained in-place sorting.

Benefits

- Correct insertion.
- Cleaner implementation.
- Better maintainability.

Justification

Insertion Sort requires placing the key after the last smaller element. Using $j + 1$ satisfies this condition and guarantees correct ordering.

4. Solution Design & Approach

Corrected & Optimized Code

```
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Shift larger elements
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert key in correct position
        arr[j + 1] = key

    return arr
```


Algorithm / Logic Flow

1. Start the program.
2. Read input array.
3. Select current element as key.
4. Shift larger elements one position right.
5. Insert key at $j + 1$.
6. Repeat for all elements.
7. Display sorted array.
8. Stop.

Output:

Output

Original Array: [5, 2, 4, 6, 1, 3]

Sorted Array: [1, 2, 3, 4, 5, 6]

=== Code Execution Successful ===

5. Clarity, Justification & Presentation

Time Complexity Analysis

Case	Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity: Auxiliary Space: **$O(1)$**

Final Conclusion

- The insertion index was incorrect.
- $\text{arr}[j] = \text{key}$ was replaced with $\text{arr}[j + 1] = \text{key}$.
- The corrected algorithm properly inserts elements after shifting.
- Time complexity remains unchanged.
- The implementation now produces correct sorted output efficiently.